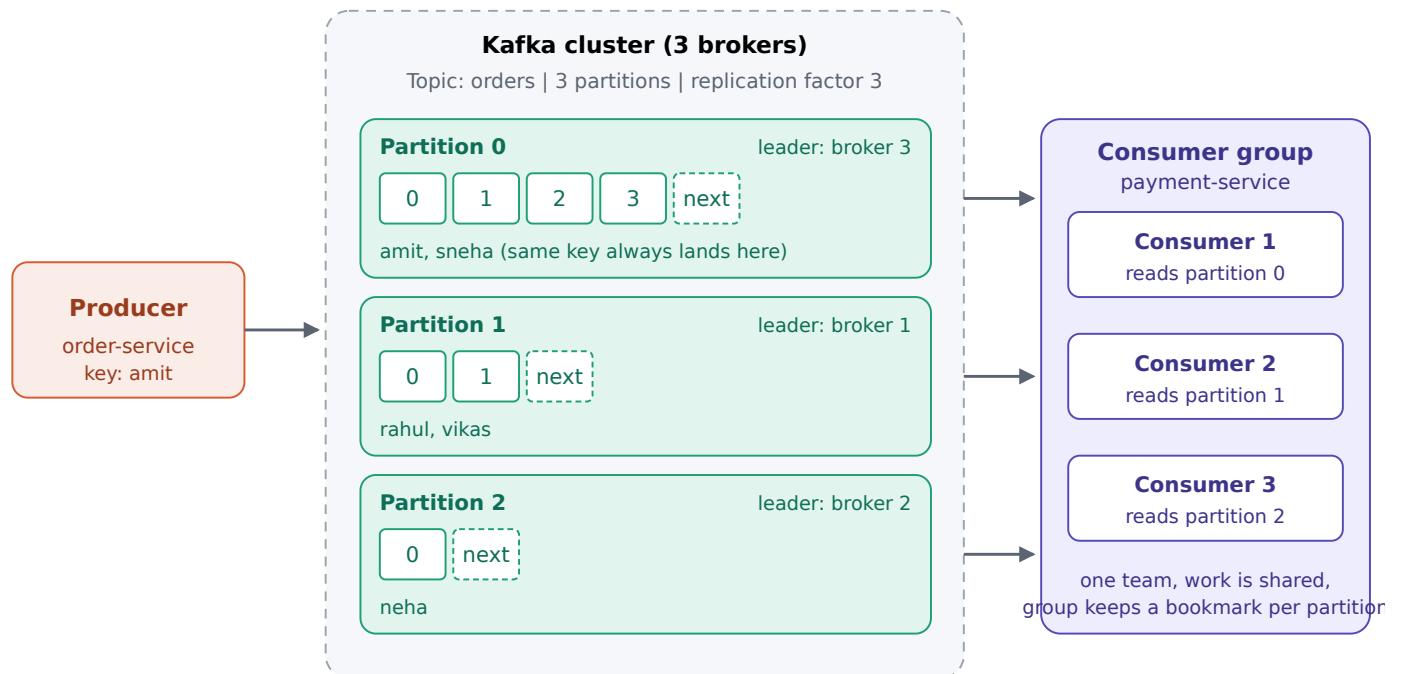


Apache Kafka: My Hands-On Notes

Everything learned by building and breaking: plain Java, 3-broker cluster, and Spring Boot. Kafka 4.0 (KRaft), JDK 21.

The whole picture in one diagram



Producer writes, the key picks the partition, each message gets an offset (line number), and each partition is read by exactly one consumer inside a group.

Real-life picture (used all through these notes): Kafka is the Zomato **order office**. A **topic** is an order register, split into **partitions** (separate notebooks) so several clerks can work at once. The **offset** is the line number in a notebook. A **consumer group** is a team (payment team, SMS team), and each team keeps its own **bookmark**.

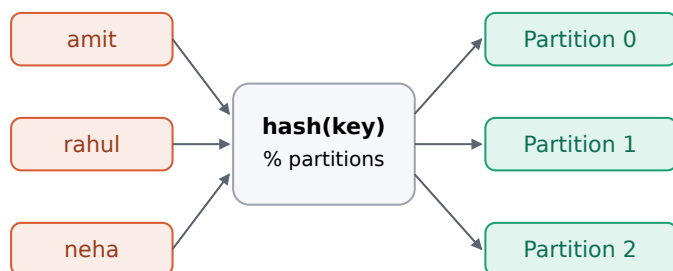
Contents

1. Core building blocks
2. Keys, partitions and ordering
3. Consumer groups and rebalance
4. Offsets, bookmark and lag
5. Commits and delivery guarantees
6. Idempotency
7. Brokers, replicas, leader and ISR
8. acks and min.insync.replicas
9. Spring Boot + Kafka
10. Retries, poison messages and DLT
11. JSON and schema changes
12. Cheat sheet, interview answer bank, commands

1 Core building blocks

Term	One-line meaning	Real life
Kafka	A system that stores messages in order and lets many apps read them at their own speed.	The order office
Broker	One Kafka server that stores messages.	One office branch
Cluster	A group of brokers working together.	All branches together
Topic	A named category of messages, e.g. <code>orders</code> .	The order register
Partition	A piece of a topic; topics are split so work can run in parallel.	Separate notebooks
Offset	The position (line number) of a message inside one partition. Never changes.	Line number
Record / message	One piece of data: a key + a value (+ optional headers).	One parcel
Key	Decides which partition the message goes to. Same key, same partition.	Address on the parcel
Value	The actual content of the message.	What's inside the parcel
Headers	Small extra labels on a message (Spring puts the DLT error reason here).	Sticky notes on the parcel
Producer	An app that writes messages to a topic.	Customer placing orders
Consumer	An app that reads messages from a topic.	A staff member
Serializer	Turns a Java object into bytes before sending (Kafka stores only bytes).	Packing a table flat
Deserializer	Turns bytes back into a Java object when reading.	Assembling it again
Retention	How long Kafka keeps messages (default 7 days). Reading does NOT delete.	CCTV recording
Batching	The producer groups messages by partition and sends them together, to be faster.	One bag per delivery area
KRaft	Kafka's built-in cluster management. Replaced Zookeeper (removed in Kafka 4).	Office runs itself

2 Keys, partitions and ordering

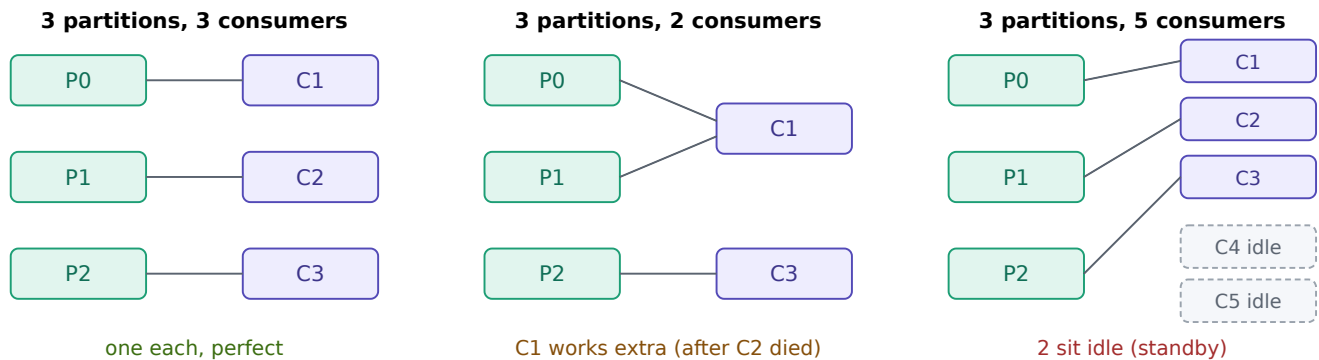


Same key, same hash, same partition, every time.

- **Order is guaranteed only inside one partition**, never across the whole topic.
- Many keys share a partition (3 counters, 1000 customers).
- Need events of one order in sequence (created, paid, delivered)? Use `orderId` as the key.
- `send()` is **asynchronous**: it puts the message in a buffer and returns. The producer **batches** messages per partition and ships them together, so receipts come back grouped by partition.

3 Consumer groups and rebalance

Main rule: inside one group, each partition is read by **exactly one** consumer. One consumer can hold several partitions.



Max useful consumers in one group = number of partitions.

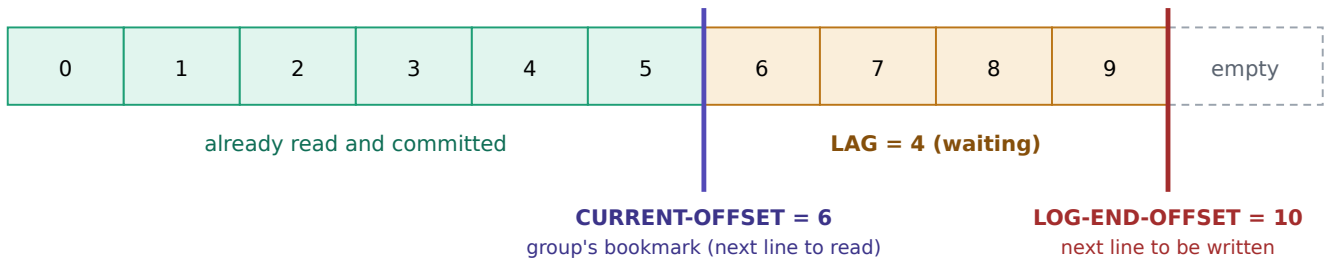
Term	One-line meaning
Consumer group	A team of consumers sharing one topic's work. Identified by <code>group.id</code> .
Rebalance	Kafka redistributes partitions when a consumer joins or leaves the group.
Idle consumer	A consumer with no partition, because there are more consumers than partitions. Acts as standby.
Heartbeat	The consumer's regular "I am alive" signal to Kafka (every 3 s by default).
Session timeout	How long Kafka waits without a heartbeat before declaring a consumer dead (<code>session.timeout.ms</code> , 45 s default).
Graceful shutdown	<code>consumer.close()</code> tells Kafka "I'm leaving" so the rebalance happens at once. A crash / red STOP button makes Kafka wait for the timeout.
Queue vs pub-sub	Inside one group, messages are shared (queue). Across groups, every group gets all messages (pub-sub).

Real life: 3 billing counters. If a cashier walks out without telling the manager, the manager waits a while ("maybe he's in the washroom"), then hands the counter to someone else. The new cashier continues from the bookmark in the register.

Interview: "When a consumer dies, the group coordinator notices missing heartbeats after `session.timeout.ms`, triggers a rebalance and gives its partitions to the others. They resume from the last committed offset, so nothing is lost, but uncommitted messages may be read again."

4 Offsets, bookmark and lag

Partition 0 of topic orders



Lag formula: $\text{LAG} = \text{LOG-END-OFFSET} - \text{CURRENT-OFFSET}$ (per partition). Total lag = sum over all partitions. Growing lag means the consumer is slower than the producer, stuck, or down.

Term	One-line meaning
Committed offset	The group's saved bookmark per partition: "we finished everything before this line". Belongs to the group , not to one consumer, so it survives rebalances and restarts.
Log-end offset	Where the next new message will be written in the partition.
Lag	How many messages are written but not yet processed by the group.
<code>auto.offset.reset</code>	Used only when the group has no bookmark . <code>earliest</code> = start from offset 0. <code>latest</code> = only new messages from now.
<code>poll()</code>	The consumer asking "any new messages?", in a never-ending loop.

What I saw in my experiments

- Console consumer **without a group** used `latest`, so it **missed** orders sent while it was off.
- Stopped the `payment-service` group, sent 4 orders: lag showed $1 + 2 + 1 = 4$. Restarted: it printed exactly those 4 and lag went to 0.
- New group `notification-service` with `earliest` read **all 23** old messages, not affected by the other group.

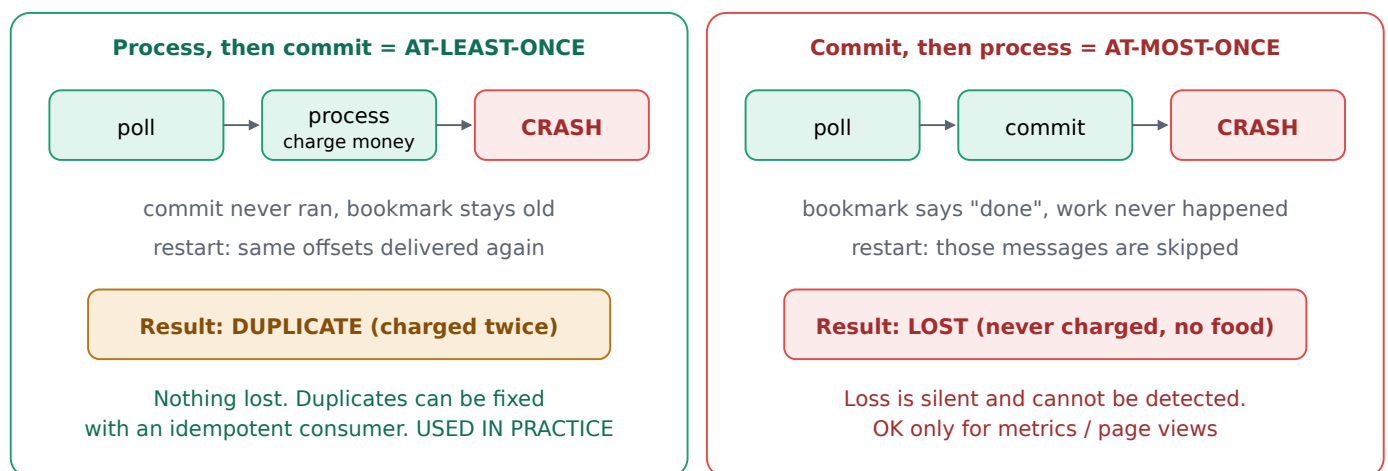
Interview: "Lag is the number of messages written to a partition that the consumer group has not processed yet: log-end offset minus committed offset. If lag keeps growing, consumers are slower than producers, stuck on a failing message, or down. I check it with `kafka-consumer-groups --describe` or monitoring dashboards."

```
docker exec -it kafka1 /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka1:19092 --describe --group payment-service
# add --members to also see idle consumers (#PARTITIONS = 0)
```

5 Commits and delivery guarantees

Term	One-line meaning
Auto commit	Kafka client saves the bookmark automatically every 5 s (<code>enable.auto.commit=true</code> , plain Java default).
Manual commit	Your code decides when to save the bookmark (<code>enable.auto.commit=false</code>).
<code>commitSync()</code>	Saves the bookmark for everything from the last <code>poll()</code> and waits until Kafka confirms.
Duplicate	The same partition + same offset processed twice. Same text at a different offset is a new message.

Key idea: *reading* a message and *saving the bookmark* are two separate steps. A crash between them decides what goes wrong.



Guarantee	How	Risk
At-most-once	Commit before processing	Messages can be lost
At-least-once	Process, then commit (Kafka + Spring default style)	Duplicates possible
Exactly-once	Kafka transactions (advanced, read-process-write inside Kafka)	More complex, slower

Which one do companies use? Almost always **at-least-once + idempotent consumer**. A lost message is usually impossible to detect or fix later; a duplicate can be detected and skipped ("is order-20 already paid?"). At-most-once is only acceptable when losing a few messages doesn't matter, like page-view counts or sensor readings.

Real life: at-least-once = cashier gives the food, then faints before ticking the register, so the next cashier serves it again. At-most-once = cashier ticks "served" first, faints before cooking, so the customer never gets food.

Interview: "It depends on when you commit. Commit after processing gives at-least-once (possible duplicates). Commit before gives at-most-once (possible loss). We use at-least-once with idempotent consumers, because duplicates can be handled but lost messages can't."

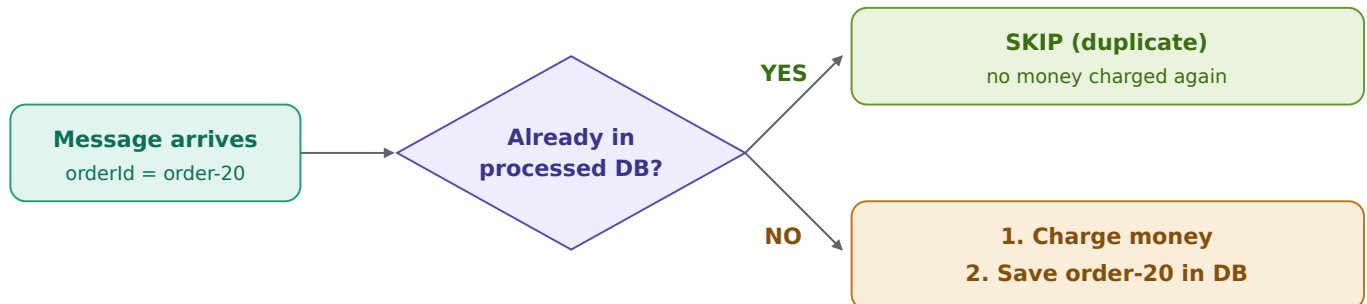
6 Idempotency

Idempotent = doing the same thing twice gives the same result as doing it once.

Idempotent: pressing the lift button for floor 5 ten times. The lift still goes to 5 once.

Not idempotent: paying ₹100 at a shop twice. You paid ₹200.

Idempotent consumer (your code)



The "processed" list must survive crashes: a DB table (unique constraint), not an in-memory HashSet.

My experiment: crashed between processing and commit. On restart, Kafka redelivered offsets 5 to 9, and the app printed `Loaded 5 already-processed orders` then `SKIPPED (duplicate)` five times. Charged only once.

Remember: Kafka guarantees "not lost". Your code guarantees "not twice".

Idempotent producer (Kafka setting)

Problem: the broker saves pay-10, but the "OK" reply is lost on the network. The producer retries, and pay-10 is written twice.

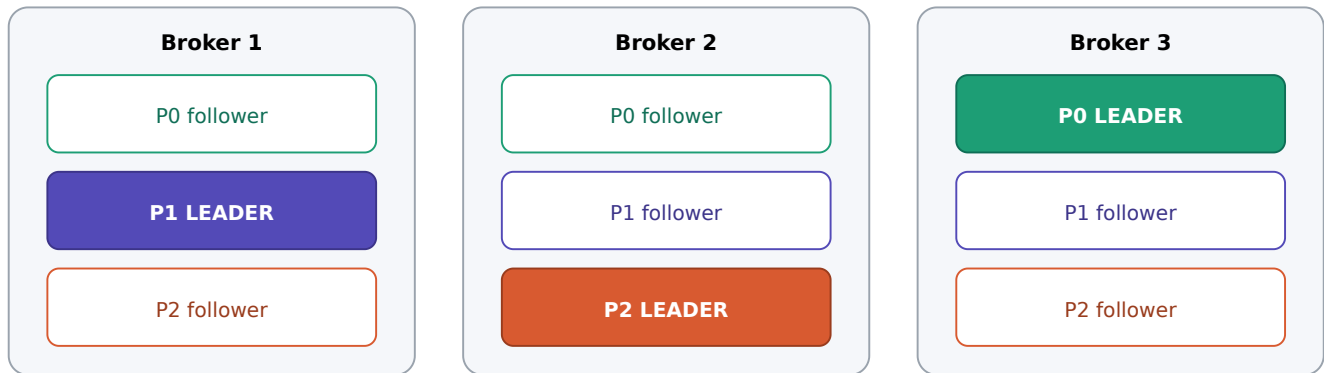
Fix: `enable.idempotence=true` (default since Kafka 3.0). Each producer gets an ID, each message a sequence number. The broker ignores a repeat of the same ID + sequence.

Real life: every courier parcel has a tracking number. If the same tracking number arrives twice, the office keeps just one.

Kind	Where	Solves
Idempotent consumer	Your code + DB	Same message delivered twice after crash / rebalance
Idempotent producer	Kafka producer config	Same message written twice because of producer retries

Interview: "Kafka consumers are at-least-once by default, so I make them idempotent: every event carries a unique ID, and before processing I check a processed-events table or rely on a unique constraint. Duplicates are skipped. On the producer side, idempotence is on by default, so retries don't create duplicate writes."

7 Brokers, replicas, leader and ISR



Replication factor 3: every partition has 3 copies on 3 brokers. Each broker leads one partition (load is balanced). Followers copy from their leader.

Term	One-line meaning
Replication factor	How many copies of each partition Kafka keeps (e.g. 3).
Replica	One copy of a partition on one broker.
Leader	The replica that handles all reads and writes for that partition. Head clerk.
Follower	A replica that only copies data from the leader. Assistant.
ISR (In-Sync Replicas)	Replicas fully caught up with the leader. Only these can become the next leader.
Replicas vs ISR	Replicas = the plan (who should have a copy). ISR = the reality (who is up to date right now).
Leader election	When a leader dies, a new leader is chosen from the ISR.
Preferred leader	First broker in the Replicas list. Kafka gives leadership back to it once it's healthy (checks about every 5 min).

What happened when I killed broker 3

Moment	Partition 0	What it shows
Before	Leader 3, Replicas 3,1,2, ISR 3,1,2	All healthy
Broker 3 stopped	Leader 1 , Replicas 3,1,2, ISR 1,2	New leader from ISR; dead broker leaves ISR, stays in Replicas
Sent pay-5, pay-6	Written by broker 1	Work continues, broker 3 misses them
Broker 3 started	Leader 3 , ISR 1,2,3	Caught up first, rejoined ISR, got leadership back

All 6 payments were still readable, including pay-5 on partition 0 offset 2. No data lost.

Interview: "When a broker dies, for every partition it led, the controller elects a new leader from the remaining in-sync replicas. The dead broker drops out of the ISR. Clients discover the new leader and continue. When it returns, it catches up from the leader, rejoins the ISR, and preferred-leader election gives back its leadership. As long as an in-sync replica exists, committed data is not lost."

8 acks and min.insync.replicas

Ack = the broker's confirmation **to the producer** that the message is **saved in Kafka**. It has nothing to do with consumers.

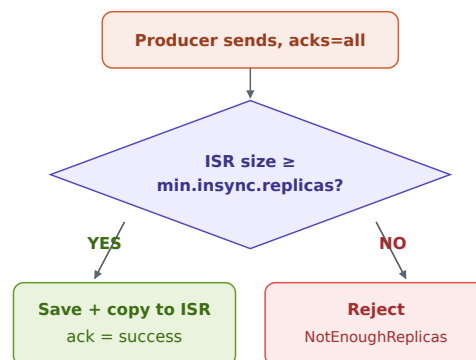
acks	Producer gets "saved" when...	Bank example	Risk
0	Never. Fire and forget.	Drop cash in a box and walk away	Loss, and you'd never know
1	The leader has saved it.	Cashier signs before copies are made	Lost if leader dies right after
all (-1)	Leader + every replica in the ISR saved it. Default in Kafka 3+.	Receipt only after all open branches copied it	Safest; a bit slower

Term	One-line meaning
<code>min.insync.replicas</code>	The minimum ISR size needed for an <code>acks=all</code> write to be accepted.
<code>NotEnoughReplicasException</code>	The error when the ISR is smaller than <code>min.insync.replicas</code> .
Retries	The producer automatically re-sends after temporary errors (you saw "2 attempts left... 0 attempts left").

Who sets what

Setting	Set on
<code>acks</code>	Producer config
<code>min.insync.replicas</code>	Topic / broker config
<code>group.id</code> , <code>commits</code>	Consumer config

`min.insync.replicas` = the minimum ISR size for an `acks=all` write to be accepted. It is checked **only** for `acks=all`, and it **never** affects reads.



My experiment: RF 3, min.insync.replicas = 2

Brokers alive	ISR	acks=all	acks=1	Consumer read
3	3	Accepted	Accepted	Works
2	2	Accepted (pay-7)	Accepted	Works
1	1	Refused (pay-8) (not stored at all), 3 retries then <code>NotEnoughReplicasException</code>	Accepted (pay-9) but only 1 copy	Still works

Golden production combo: replication factor 3 + `acks=all` + `min.insync.replicas=2`. Survives 1 broker failure and keeps accepting writes. With 2 brokers down, writes are refused instead of risked.

Interview: "acks is a producer setting and `min.insync.replicas` is a topic setting. With `acks=all`, the leader accepts a write only if the ISR has at least `min.insync.replicas` members, and replies after all of them have it. Otherwise the producer gets `NotEnoughReplicasException`. A clear error is better than a fake-safe receipt."

9 Spring Boot + Kafka

Plain Java (what I wrote)	Spring Boot
Properties + props.put(...)	application.yml under spring.kafka
new KafkaProducer<>(props)	KafkaTemplate (injected)
producer.send(record, callback)	kafkaTemplate.send(topic, key, value).whenComplete(...)
new KafkaConsumer + subscribe()	@KafkaListener(topics = "orders")
while(true) { poll(); }	Spring's listener container runs the loop in the background
commitSync() by hand	Spring turns auto commit off and commits after your listener returns (at-least-once)
try/catch yourself	DefaultErrorHandler: retries, then recoverer (DLT)

```
spring:
  kafka:
    bootstrap-servers: localhost:9092,localhost:9094,localhost:9095
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
    consumer:
      group-id: spring-payment-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "com.amit.kafkaspring.model"
```

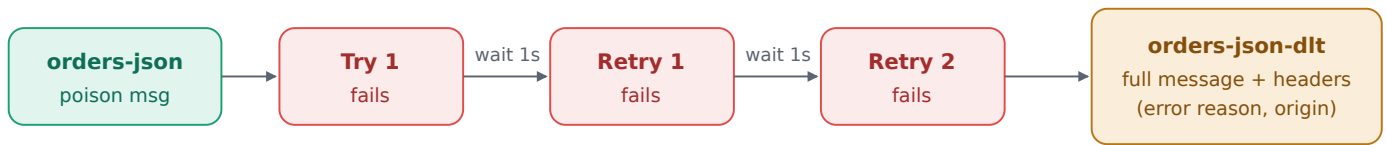
```
@PostMapping("/orders")
public Order place(...) {
    Order o = new Order(UUID..., customer, item,
price);
    kafkaTemplate.send("orders-json", o.customer(),
o);
    return o;
}
```

```
@KafkaListener(topics = "orders-json")
public void onOrder(ConsumerRecord<String, Order>
r) {
    Order o = r.value(); // already an object
    ... process ...
}
```

Things I tripped on (and learned from)

- **"No group.id found"**: application.yml was not in src/main/resources, so Spring never read it. Wrong place = invisible settings.
- **acks = -1** in the logs is the same as **acks=all**. Producer settings print only on the first send (producer is created lazily).
- **GOT printed before SENT**: listener and producer callback run on different threads, so print order isn't guaranteed.
- **Auto topic creation** silently made **orders** with 1 partition and RF 1 (a running consumer re-created it right after I deleted it). Create topics deliberately; many companies set **auto.create.topics.enable=false**.
- Logs also confirmed: **enable.idempotence = true**, **enable.auto.commit = false** (Spring commits itself).

10 Retries, poison messages and DLT



Next message on the same partition (idli) waits until the poison one is sent to the DLT, then continues.

	Spring default	With DLT config
Tries	10, instantly	3, 1 s apart
After giving up	Only logged: lost	Saved in DLT

```
@Bean
DefaultErrorHandler
errorHandler(KafkaTemplate<String,?> t) {
    var dlt = new DeadLetterPublishingRecoverer(t,
        (rec, ex) -> new TopicPartition(
            rec.topic() + "-dlt", rec.partition()));
    return new DefaultErrorHandler(dlt,
        new FixedBackOff(1000L, 2));
}
```

Term	Meaning
Poison message	Fails every time it's processed.
Backoff	Wait time between retries; gives temporary problems time to heal.
DLT	A normal topic, used by agreement for failed messages. Nothing is lost; can be fixed and replayed.

A failing message blocks **reading** of its own partition only. Writing continues, and other partitions keep working. Lag grows on that partition while it's retried.

11 JSON and schema changes

Kafka stores only bytes. `JacksonJsonSerializer` turns `Order` into JSON bytes; `JacksonJsonDeserializer` turns them back (Spring Kafka 4 names; old `JsonSerializer` is deprecated). **Trusted packages** = only create objects from classes in your own package.

Producer change	Effect on old consumer	Handle it by
Add optional field	Safe: unknown field ignored	Nothing (backward compatible)
Remove / rename field	Silent bug: <code>price</code> becomes 0 / null	Validate required fields, invalid → DLT
Change field type	Deserialization fails every time	<code>ErrorHandlingDeserializer</code> → DLT

Interview: "Only additive, optional changes are safe. For breaking changes, version the event or use a new topic like `orders-v2`. Best practice is a Schema Registry with Avro / Protobuf / JSON Schema, which rejects incompatible schemas before the producer can publish."

12 One-page cheat sheet

- 1 Order is guaranteed **only inside one partition**. Same key → same partition.
- 2 **Max parallel consumers in a group = number of partitions**. Extra consumers sit idle.
- 3 Reading **does not delete**. Each group has its own bookmark. Messages expire by retention (7 days default).
- 4 **Lag = log-end offset – committed offset**. Growing lag = slow, stuck or dead consumers.
- 5 **Process, then commit** (at-least-once) + **idempotent consumer**. Kafka: "not lost". Your code: "not twice".
- 6 **RF 3 + acks=all + min.insync.replicas=2** survives one broker failure with no data loss.
- 7 New leader is always picked **from the ISR**. Replicas = plan, ISR = reality.
- 8 Failing messages: **retry with backoff, then DLT**. Never let them silently disappear.

Default numbers worth remembering

Setting	Default
<code>acks</code>	all (-1), since Kafka 3.0
<code>enable.idempotence</code>	true, since Kafka 3.0
<code>enable.auto.commit</code>	true (plain Java); Spring sets false
<code>auto.commit.interval.ms</code>	5000 (5 s)
<code>auto.offset.reset</code>	latest
<code>session.timeout.ms</code>	45000 (45 s)
<code>heartbeat.interval.ms</code>	3000 (3 s)
<code>log.retention.hours</code>	168 (7 days)
Spring retries	9 retries, no wait (10 tries)
Auto topic creation	1 partition, RF 1 (by default)

Quick interview answers

Question	Short answer
Consumer dies?	Rebalance; others take its partitions from the last commit.
Broker dies?	New leader from ISR; clients follow; no loss if ISR copy exists.
Why duplicates?	Crash after processing, before commit.
Avoid duplicates?	Idempotent consumer: unique ID + processed table.
Keep order per order?	Use <code>orderId</code> as key.
<code>send()</code> sync?	Async; batch + callback. <code>.get()</code> blocks.
More consumers than partitions?	Extras idle; add partitions to scale.
Poison message?	Retry with backoff, then DLT.
Schema change?	Additive only, version, Schema Registry.
Queue or pub-sub?	Queue inside a group, pub-sub across groups.

My summary, in my own words

- Producer sends messages to a topic
- A topic is split into partitions
- Each message in a partition has an offset (its line number)
- The key decides the partition (same key → same partition → order kept)
- In a group, each partition is read by only one consumer
- 3 partitions, 3 consumers → 1 each
- 4 partitions, 5 consumers → 4 work, 1 sits idle
- 4 partitions, 3 consumers → one consumer handles 2
- Redistributing partitions on join / leave = rebalance
- The group's committed offset (bookmark) survives a rebalance
- Lag = messages written – messages the group has read
- acks=0: producer gets no info; acks=1: info when the leader saved it; acks=all: info when leader + ISR saved it

Interview answer bank

Full answers, ready to say out loud. Each one is backed by an experiment you ran.

Question	Answer
What is lag?	"Lag is the number of messages written to a partition that the consumer group has not processed yet: log-end offset minus committed offset. If it keeps growing, consumers are slower than producers, stuck, or down."
What happens when a consumer in a group dies?	"Kafka detects it's gone, triggers a rebalance, and assigns its partitions to the remaining consumers in the group. The new owner starts from the last committed offset of that partition, so no messages are lost. Messages processed but not yet committed may be read again, which is why consumers should handle duplicates."
How does Kafka detect a dead consumer?	"Through heartbeats. If the coordinator doesn't get a heartbeat within <code>session.timeout.ms</code> , it removes the consumer and rebalances its partitions to the others. A graceful <code>close()</code> avoids the wait."
3 partitions and 5 consumers in one group?	"Only 3 consumers get a partition. The other 2 stay idle. So the number of partitions is the maximum parallelism of a consumer group. If you want more consumers working in parallel, you must increase the partitions."
Is <code>send()</code> sync or async?	"Asynchronous. It adds the record to a buffer and returns immediately. The producer batches records per partition and sends them in the background. You get the result through a callback or the returned Future. Calling <code>.get()</code> makes it blocking, but that's slower."
Why might a consumer process the same message twice?	"If it processes a message but crashes before committing the offset, then after restart or rebalance it resumes from the last committed offset and reads that message again. That's why consumers should be idempotent: processing the same message twice should be safe."
How do you avoid duplicate processing?	"Kafka consumers give at-least-once delivery by default. If a consumer crashes after processing but before committing, messages get redelivered. So I make the consumer idempotent: each event has a unique ID, and before processing I check whether it's already handled (for example, a unique constraint or a processed-events table in the DB). Duplicates are then safely skipped."
At-least-once vs at-most-once?	"It depends on when you commit the offset. Commit after processing gives at-least-once: a crash causes redelivery, so duplicates are possible. Commit before processing gives at-most-once: a crash can lose messages. In practice we use at-least-once with idempotent consumers, because duplicates can be handled but lost messages can't."

What happens when a broker dies?	"For every partition where that broker was the leader, the controller elects a new leader from the remaining in-sync replicas. The dead broker is removed from the ISR of all its partitions. Producers and consumers automatically discover the new leader and continue. As long as an in-sync replica exists, no committed data is lost."
How do you make sure Kafka doesn't lose messages?	"Use replication factor 3, <code>acks=all</code> on the producer, and <code>min.insync.replicas=2</code> on the topic. Then every acknowledged write exists on at least 2 brokers, so we survive one broker failure. If too many brokers are down, the producer gets <code>NotEnoughReplicasException</code> instead of silently losing data. Plus idempotent consumers with <code>commit-after-processing</code> on the consumer side."
acks vs min.insync.replicas?	" <code>acks</code> is a producer setting, and <code>min.insync.replicas</code> is a topic or broker setting. When a producer sends with <code>acks=all</code> , the leader accepts the write only if the ISR has at least <code>min.insync.replicas</code> members. Otherwise it returns <code>NotEnoughReplicasException</code> . Consumers are not affected by either setting."
How do you handle messages that keep failing?	"In Spring Kafka, I configure a <code>DefaultErrorHandler</code> with a backoff, for example 3 attempts 1 second apart, for temporary failures. After retries are exhausted, a <code>DeadLetterPublishingRecoverer</code> sends the record to a dead letter topic, with headers like the exception message and original partition and offset. That way the partition isn't blocked forever, nothing is silently lost, and the team can inspect, fix, and replay DLT messages later."
Can you increase partitions of an existing topic?	"Yes, with <code>kafka-topics --alter --partitions</code> , but never decrease them. The key-to-partition mapping changes, so the same key may go to a different partition afterwards, which breaks ordering for that key during the transition."
Why disable auto topic creation?	"A typo or an early write creates a topic with default settings, often 1 partition and replication factor 1: no parallelism and no fault tolerance. Topics should be created deliberately with the right partitions, replication factor and <code>min.insync.replicas</code> ."
JSON in Spring Kafka?	"Kafka stores only bytes. We use a JSON serializer on the producer and a JSON deserializer on the consumer, with trusted packages configured, so services exchange typed objects like <code>Order</code> as JSON."
Producer changed the JSON structure?	"Adding optional fields is safe; the consumer ignores unknown fields. Removing or renaming fields gives silent nulls, so I validate required fields and route invalid messages to a DLT. Type changes break deserialization, so I use <code>ErrorHandlingDeserializer</code> to send them to the DLT. To prevent it: compatibility rules between teams, versioned events or a new topic, and ideally a Schema Registry with Avro or Protobuf."

Commands I used

Inside Docker use `kafka1:19092`. From Java on the laptop use `localhost:9092,localhost:9094,localhost:9095`.

Cluster

```
docker compose up -d          # start 3 brokers (in the folder with docker-compose.yml)
docker ps                     # see kafka1, kafka2, kafka3
docker stop kafka3            # kill a broker
docker start kafka2 kafka3    # bring them back
```

Topics

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:19092 \
  --create --topic orders --partitions 3 --replication-factor 3 --config min.insync.replicas=2

docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:19092 --describe --
topic orders
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:19092 --delete --topic
orders

docker exec -it kafka1 /opt/kafka/bin/kafka-configs.sh --bootstrap-server kafka1:19092 \
  --alter --entity-type topics --entity-name payments --add-config min.insync.replicas=2
```

Produce and consume

```
docker exec -it kafka1 /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server kafka1:19092 \
  --topic orders --property parse.key=true --property key.separator=:
#   then type:  amit:order-1 pizza           (add --producer-property acks=1 to test acks)

docker exec -it kafka1 /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka1:19092 \
  --topic orders --group payment-service --from-beginning \
  --property print.key=true --property print.partition=true --property print.offset=true
```

Consumer groups and lag

```
docker exec -it kafka1 /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka1:19092 \
  --describe --group payment-service           # PARTITION, CURRENT-OFFSET, LOG-END-OFFSET, LAG
docker exec -it kafka1 /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka1:19092 \
  --describe --group payment-service --members # shows idle consumers too
```

Spring app test

```
curl -X POST "http://localhost:8080/orders?customer=amit&item=chai&price=20"
curl -X POST "http://localhost:8080/orders?customer=rahul&item=poison&price=50" # goes to DLT
```

How to read `--describe --topic`

Partition: 0	Leader: 3	Replicas: 3,1,2	Isr: 3,1,2
	head clerk	who SHOULD have	who IS up to date
		a copy (plan)	right now (reality)

Troubleshooting notes: after pressing STOP on a consumer, wait about 45 s before expecting its partitions to move (session timeout). `UnknownHostException: kafka2` just means that broker is stopped. SLF4J warnings are harmless.